□法速查手册（CSP 300 分版）

> □手册按 **CSP** □证 **300** 分目标裁剪，聚焦 T1～T3 常用算法。
>
> 每章列出「识别关键词 → 子类型分析 → 模板代码 → 例题」，所有代码使用 **C++17**。
>
> ⚠️ KMP 字符串匹配、线段树、Bellman-Ford 等在 CSP T1–T3 基□不考，□手册已省略（目标 400+ 分时再学）。

目录

编号	题型关键词	算法
1	按规则操作、逐步推导	暴力枚举 / 模拟
2	有序序列中找值、最小化最大值	二分查找 / 二分答案
3	局部最优 → 全局最优、区间选择	贪心算法
3.5	连续子数组、两端夹逼、链表节点	双指针
4	所有可能、路径、子集、排列	DFS / 回溯
5	最短步数、层次扩展、感染扩散	BFS
6	重叠子问题、最优子结构	动态规划（DP）
7	有向/无向图、路径、联通性	图论（基础）
8	区间求和/更新	前缀和 / 差分
8.5	二维数组变换、旋转、遍历	矩阵操作（重点技巧）
9	下一个更大元素、滑动窗口最值	单调栈 / 单调队列
10	集合合并、判断联通	并查集
10.5	边界总写错、模板不熟	边界易错点 + 常用函数模板
11	容器**/API** 日常使用（非算法）	**STL** 日常速查（非算法）

手册题目总览（题目 → 思路 → □法）

> 下面把□手册的全部例题集中做一页速览：每题给出一句话思路和对应算法，便于考前快速回顾。

题号	题目	大致思路	使用算法
1	约瑟夫问题	用 `vector` 按报数规则循环删除元素直到剩 1 人	模拟
2	字符串压缩	逐字符扫描，记录前一个是否空格并执行替换	字符串模拟
3	排序数组查找	在有序数组上二分定位目标值	二分查找
4	砍树	对答案 H 二分，`check(H)` 统计可收集木材量	二分答案
5	区间调度	按右端点排序，优先选择结束最早区间	贪心（区间）
6	分发糖果	左右各扫一遍，合并相邻约束下的最小糖果数	贪心（双向扫描）
A	最长无重复子串	维护窗口内无重复字符，右扩左缩	双指针（变长滑窗）
B	最短子数组和≥target	右指针扩张累加，满足条件后左指针收缩	双指针（变长滑窗）
C	有序数组两数之和	左右夹逼，根据和与目标比较移动指针	双指针（前后夹逼）
D	链表中点	快指针走 2 步，慢指针走 1 步	双指针（快慢）
7	全排列	回溯枚举每个位置可选数字并撤销选择	DFS / 回溯
8	岛屿数量	遍历网格，遇到陆地即 DFS 淹没整块连通域	网格 DFS
9	迷宫最短路	BFS 分层扩展，首次到达终点即最短步数	BFS（单源）

题号	题目	大致思路	使用算法
10	腐烂橘子	初始腐烂点同时入队，按层扩散统计分钟数	BFS（多源）
11	LIS	`dp[i]` 表示以 i 结尾的最优长度，枚举前驱转移	线性 DP
12	0/1 背包	容量倒序枚举，保证每件物品最多使用一次	背包 DP（0/1）
12.5	完全背包	容量正序枚举，允许同一物品重复使用	背包 DP（完全）
13	LCS	二维 `dp[i][j]` 比较末位是否相等后转移	二维 DP
14	课程表	用入度队列做拓扑排序，判断是否能处理完所有点	图论（拓扑排序）
15	单源最短路	最小堆维护当前最短点，持续松弛边权	图论（Dijkstra）
16	区间求和	预处理前缀和，查询时两次前缀相减	前缀和
17	区间批量加	差分数组打标记，最后前缀还原	差分
18	每日温度	单调栈维护未匹配下标，遇到更大值时结算答案	单调栈
19	滑动窗口最大值	单调队列维护窗口候选最大值下标	单调队列
20	朋友圈	并查集合并关系，统计连通分量个数	并查集

快速题型判断流程图

```
□到题目
    │
    ├─ 数据范围极小（n≤1000）或按规则一步步操作？ ──→ 暴力模拟
    │
```

```
    ├─ 有序序列中找边界，或答案具有单调性?  ──→  二分查找 / 二分答案
    │
    ├─ 连续子数组/子串，或有序数组两端夹逼?  ──→  双指针
    │
    ├─ 求最多/最少，局部最优可推全局最优?  ──→  贪心
    │
    ├─ 网格/图/树，求联通分量或路径?
    │       ├─ 求最短步数（无权）？  ──→ BFS
    │       └─ 枚举所有路径/状态?  ──→ DFS / 回溯
    │
    ├─ 最优解，且有重叠子问题?  ──→  动态规划
    │       ├─ 序列最值  ──→  线性DP
    │       ├─ 每件选一次  ──→  0/1 背包（逆序）
    │       └─ 每件无限选  ──→  完全背包（正序）
    │
    ├─ 区间求和（静态）──→  前缀和；区间批量更新  ──→  差分
    ├─ 二维数组旋转/转置/螺旋遍历?  ──→  矩阵操作
    │
    ├─ 有向图无环，求执行顺序?  ──→  拓扑排序
    ├─ 带权图，求单源最短路?  ──→  Dijkstra
    ├─ 下一个更大/更小元素?  ──→  单调栈
    ├─ 滑动窗口最值（窗口较大时）？  ──→  单调队列
    └─ 集合合并/联通判断?  ──→  并查集
```

---

1. 暴力枚举 / 模拟

识别关键词

- 题目描述像「规则说明书」，一步一步按规则执行
- 数据范围很小（$n \le 1000$ 甚至 $n \le 100$）
- 按某个公式逐一计算每个元素

子类型分析

子类型	特征	例子
纯模拟	按题目步骤一步步执行，无需优化	约瑟夫问题、游戏回合
字符串模拟	逐字符处理、字符串变换、统计	压缩字符串、替换字符
数学模拟	按公式计算（均值、方差、进制转换）	Z-score 归一化、混合进制解码
分段枚举	区间分段后各段单独计算，将 O(N) 优化为 O(n)	序列查询（范围 N 很大，节点数 n 很小）

例题 1：约瑟夫问题（纯模拟）

n 个人围圈，数到 m 淘汰，求最后剩下的人的编号。（$n \le 100$）

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<int> v;
    for (int i = 1; i <= n; i++) v.push_back(i);

    int cur = 0;
    while (v.size() > 1) {
        cur = (cur + m - 1) % v.size();
        v.erase(v.begin() + cur);
        if (cur == (int)v.size()) cur = 0;
    }
    cout << v[0] << endl;
}
```

例题 2：字符串压缩（字符串模拟）

将连续空格替换为下划线，字母转大写。

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    string s;
    getline(cin, s);
    string res;
    bool lastSpace = false;
    for (char c : s) {
        if (c == ' ') {
            if (!lastSpace) res += '_';
            lastSpace = true;
        } else {
            res += toupper(c);
            lastSpace = false;
        }
    }
    cout << res << endl;
}
```

> 📝 **CCF CSP** 真题：202209-1 如此编码（混合进制解码）· 202203-1 归一化处理（数学模拟）· 202103-1 序列查询（分段枚举）

---

2. 二分查找 / 二分答案

识别关键词

- 在有序数组中找某个值或满足条件的边界
- 答案具有单调性：可行与不可行之间有一个分界点
- 题目问「最小的最大值」「最大的最小值」

子类型分析

子类型	使用场景	关键 API / 写法
查找特定值	有序数组中找 target 是否存在	`binary_search(v.begin(),v.end(),x)`
找左边界	第一个 ≥ target 的位置	`lower_bound(v.begin(),v.end(),x)`
找右边界	最后一个 ≤ target 的位置	`upper_bound(v.begin(),v.end(),x)-1`
二分答案	答案范围已知，写 `check()` 验证单调性	自写 `while(lo<=hi)` 框架

例题 3：在排序数组中查找元素

```
int search(vector<int>& nums, int target) {
    int lo = 0, hi = nums.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] == target) return mid;
        if (nums[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

例题 4：砍树（二分答案）

有 n 棵树，高度 h[i]，截到高度 H 后收集多余木材，要求总量 ≥ M，求最大 H。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n; long long M;
    cin >> n >> M;
    vector<long long> h(n);
    long long maxH = 0;
    for (int i = 0; i < n; i++) { cin >> h[i]; maxH = max(maxH, h[i]); }
```

```
    auto check = [&](long long H) {
        long long total = 0;
        for (long long x : h) if (x > H) total += x - H;
        return total >= M;
    };

    long long lo = 0, hi = maxH, ans = 0;
    while (lo <= hi) {
        long long mid = lo + (hi - lo) / 2;
        if (check(mid)) { ans = mid; lo = mid + 1; }
        else hi = mid - 1;
    }
    cout << ans << endl;
}
```

> 📝 **CCF CSP** 真题：202303-2 垦田计划（二分最小开发天数）

---

3. 贪心□法

识别关键词

- 每次做出局部最优选择，最终得到全局最优
- 「尽可能多」「最少操作次数」「区间覆盖/选取」
- 排序后从一端开始选取

子类型分析

子类型	贪心策略	经典例子
区间调度	按右端点升序排序，优先选结束最早的	最多不重叠区间、会议室安排
区间覆盖	按左端点排序，在能到达的区间中选右端点最远的	最少区间覆盖 [s, t]
排序贪心	按某规则排序后线性扫描	分糖果、任务分配
前缀最大/最小贪心	分析每个位置的必要贡献	数组推导（最大和/最小和）

例题 5：区间调度（最多不重叠区间）

```
int eraseOverlapIntervals(vector<vector<int>>& intervals) {
    sort(intervals.begin(), intervals.end(),
         [](auto& a, auto& b){ return a[1] < b[1]; }); // 按右端点升序
    int count = 0, end = INT_MIN;
    for (auto& iv : intervals) {
        if (iv[0] >= end) { end = iv[1]; count++; }
```

```
    }
    return (int)intervals.size() - count;
}
```

例题 6：分发糖果（排序贪心）

评分高的孩子必须比相邻孩子获得更多糖果，求最少总糖果数。

```
int candy(vector<int>& ratings) {
    int n = ratings.size();
    vector<int> c(n, 1);
    for (int i = 1; i < n; i++)
        if (ratings[i] > ratings[i-1]) c[i] = c[i-1] + 1;
    for (int i = n-2; i >= 0; i--)
        if (ratings[i] > ratings[i+1]) c[i] = max(c[i], c[i+1] + 1);
    return accumulate(c.begin(), c.end(), 0);
}
```

> 📝 **CCF CSP** 真题：202109-1 数组推导（贪心分析前缀最大值）

3.5 双指针（Two Pointers）⭐ CSP 高频

识别关键词

- 在连续数组/字符串中找满足条件的最长/最短子数组
- 有序数组中找两个元素使其满足某个条件（和、差）
- 链表中找中点、检测环、删除倒数第 k 个节点

子类型分析

子类型	使用场景	指针移动方式
变长滑动窗口	找最长/最短连续子数组，条件随窗口大小单调变化	右指针先扩张；不满足时左指针收缩
定长滑动窗口	固定大小 k 的窗口，统计或求最值	两指针同时右移，始终保持间距 k
前后指针（夹逼）	有序数组中找满足条件的两个数	左从头，右从尾，根据比较结果各自移动
快慢指针	链表判环、找中点、找倒数第 k 个节点	快指针每步走 2 格，慢指针走 1 格

变长滑动窗口通用框架（以"找满足条件的最短子数组"为例）：

```
int left = 0;
for (int right = 0; right < n; right++) {
```

```
    // 1. 将 nums[right] 加入窗口（更新 sum、频次表等）
    // 2. 当窗口满足条件时，收缩左边界（找更短的满足条件的窗口）
    while (/* 例如：sum >= target */) {
        // 更新答案（当前窗口 [left, right] 满足条件）
        ans = min(ans, right - left + 1);
        // 将 nums[left] 移出窗口，左边界右移
        left++;
    }
    // 若是找最长子数组，则在 while 之外（不满足时）更新答案
}
```

例题 **A**：最长无重复字符子串（变长滑动窗口）

```
int lengthOfLongestSubstring(string s) {
    unordered_map<char, int> last; // 字符→上次出现位置
    int ans = 0, left = 0;
    for (int right = 0; right < (int)s.size(); right++) {
        if (last.count(s[right]) && last[s[right]] >= left)
            left = last[s[right]] + 1; // 左边界跳到重复字符的右边
        last[s[right]] = right;
        ans = max(ans, right - left + 1);
    }
    return ans;
}
```

复杂度：$O(n)$

例题 **B**：长度最小的子数组（变长滑动窗口）

找元素和 $\geq target$ 的最短连续子数组长度。

```
int minSubArrayLen(int target, vector<int>& nums) {
    int left = 0, sum = 0, ans = INT_MAX;
    for (int right = 0; right < (int)nums.size(); right++) {
        sum += nums[right];
        while (sum >= target) {          // 满足条件时收缩左边界
            ans = min(ans, right - left + 1);
            sum -= nums[left++];
        }
    }
    return ans == INT_MAX ? 0 : ans;
}
```

复杂度：$O(n)$

例题 **C**：两数之和（有序数组，前后指针夹逼）

有序数组中找两数之和等于 $target$，返回下标（从 1 开始）。

```
vector<int> twoSum(vector<int>& numbers, int target) {
    int left = 0, right = (int)numbers.size() - 1;
```

```
    while (left < right) {
        int sum = numbers[left] + numbers[right];
        if (sum == target) return {left + 1, right + 1};
        if (sum < target) left++;  // 和太小，左指针右移增大和
        else right--;              // 和太大，右指针左移减小和
    }
    return {};
}
```

复杂度：O(n)，无需额外空间

例题 D：链表的中间节点（快慢指针）

```
struct ListNode { int val; ListNode* next; };

ListNode* middleNode(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;       // 走 1 步
        fast = fast->next->next; // 走 2 步
    }
    return slow; // fast 到尾时，slow 恰好在中间
}
```

复杂度：O(n)，只需一次遍历

---

4. DFS / 回溯

识别关键词

- 枚举所有路径、子集、全排列
- 走迷宫、找联通分量
- 「所有可能的方案」

子类型分析

子类型	适用场景	关键技巧
网格 **DFS**	联通分量、岛屿数量、路径存在性	四方向移动，用 visited[][] 或修改原格防重
全排列枚举	所有不重复排列	`used[]` 数组防止重复选取，回溯时撤销
子集/组合枚举	所有子集、组合总和	从 `start` 位置开始选，避免重复组合
剪枝优化	提前终止不合法分支，大幅减少搜索量	加 `if (当前已不可能达到目标) return`

例题 7：全排列

```
vector<vector<int>> res;
vector<int> path;
vector<bool> used;

void dfs(vector<int>& nums) {
    if (path.size() == nums.size()) { res.push_back(path); return; }
    for (int i = 0; i < (int)nums.size(); i++) {
        if (used[i]) continue;
        used[i] = true; path.push_back(nums[i]);
        dfs(nums);
        path.pop_back(); used[i] = false; // 撤销选择（回溯）
    }
}
```

例题 8：岛屿数量（网格 DFS）

```
int dx[] = {0,0,1,-1}, dy[] = {1,-1,0,0};

void dfs(vector<vector<char>>& grid, int x, int y) {
    int m = grid.size(), n = grid[0].size();
    if (x<0||x>=m||y<0||y>=n||grid[x][y]!='1') return;
    grid[x][y] = '0'; // 标记已访问
    for (int d = 0; d < 4; d++) dfs(grid, x+dx[d], y+dy[d]);
}

int numIslands(vector<vector<char>>& grid) {
    int cnt = 0;
    for (int i = 0; i < (int)grid.size(); i++)
        for (int j = 0; j < (int)grid[0].size(); j++)
            if (grid[i][j]=='1') { dfs(grid,i,j); cnt++; }
    return cnt;
}
```

5. BFS

识别关键词

- 求最短步数（无权图）
- 层次遍历（树的层序）
- 感染/扩散问题（从多个源点同时扩散）

子类型分析

子类型	使用场景	关键特征
单源 **BFS**	从单一起点出发，求到各点的最短步数	queue + visited[]，第一次到达即最短路
多源 **BFS**	多个起点同时扩散（病毒传播、腐烂橘子）	把所有源点一起入队作为"第 0 层"
0-1 BFS	边权只有 0 和 1 时的最短路	deque：0 权边 push_front，1 权边 push_back

BFS 通用框架：

```
queue<State> q;
visited[start] = true;
q.push(start);
int step = 0;
while (!q.empty()) {
    int sz = q.size();
    while (sz--) {
        auto cur = q.front(); q.pop();
        // neighbors(cur): 返回 cur 的所有邻居节点
        // 网格题中替换为四方向遍历; 图题中替换为邻接表遍历
        for (auto next : neighbors(cur)) {
            if (!visited[next]) {
                visited[next] = true;
                q.push(next);
            }
        }
    }
    step++;
}
```

例题 9：迷宫最短路（单源 BFS）

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<string> grid(n);
    for (auto& row : grid) cin >> row;

    int sx, sy, ex, ey;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
            if (grid[i][j]=='S') { sx=i; sy=j; }
            if (grid[i][j]=='E') { ex=i; ey=j; }
        }
```

```
    int dx[] = {0,0,1,-1}, dy[] = {1,-1,0,0};
    vector<vector<int>> dist(n, vector<int>(m, -1));
    queue<pair<int,int>> q;
    dist[sx][sy] = 0; q.push({sx, sy});

    while (!q.empty()) {
        auto [x, y] = q.front(); q.pop();
        for (int d = 0; d < 4; d++) {
            int nx=x+dx[d], ny=y+dy[d];
            if (nx>=0&&nx<n&&ny>=0&&ny<m&&grid[nx][ny]!='#'&&dist[nx][ny]==-1) {
                dist[nx][ny] = dist[x][y] + 1;
                q.push({nx, ny});
            }
        }
    }
    cout << dist[ex][ey] << endl;
}
```

□□ 10□□□□□□□□□ BFS□

```
int orangesRotting(vector<vector<int>>& grid) {
    int m = grid.size(), n = grid[0].size();
    queue<pair<int,int>> q;
    int fresh = 0;
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++) {
            if (grid[i][j] == 2) q.push({i,j}); // □□□□□□□□□□
            if (grid[i][j] == 1) fresh++;
        }
    if (fresh == 0) return 0;

    int dx[] = {0,0,1,-1}, dy[] = {1,-1,0,0};
    int minutes = 0;
    while (!q.empty() && fresh > 0) {
        minutes++;
        int sz = q.size();
        while (sz--) {
            auto [x, y] = q.front(); q.pop();
            for (int d = 0; d < 4; d++) {
                int nx=x+dx[d], ny=y+dy[d];
                if (nx>=0&&nx<m&&ny>=0&&ny<n&&grid[nx][ny]==1) {
                    grid[nx][ny] = 2; fresh--; q.push({nx,ny});
                }
            }
        }
    }
    return fresh == 0 ? minutes : -1;
}
```

6. □□□□□DP□

识别关键词

- 「最多/最少/最长/最大」且有重叠子问题
- 当前状态依赖之前的状态
- 「能否达到」「方案数」

子类型分析

子类型	状态定义	核心转移	代表题
线性 DP	`dp[i]` = 前 i 个的答案	`dp[i] = max(dp[i-1]+a[i], a[i])`	最大子数组和、LIS
0/1 背包	`dp[j]` = 容量 j 时最大价值（每件选一次）	逆序枚举：`dp[j] = max(dp[j], dp[j-w]+v)`	经典背包
完全背包	`dp[j]` = 容量 j 时最大价值（每件无限选）	正序枚举：`dp[j] = max(dp[j], dp[j-w]+v)`	零钱兑换
二维 DP	`dp[i][j]` = 两序列前 i、j 的答案	`if a[i]==b[j]: dp[i][j]=dp[i-1][j-1]+1`	LCS

> 💡 区分 0/1 背包 vs 完全背包：逆序遍历容量 → 每件只能用一次；正序遍历容量 → 每件可无限用。

DP 三步法

1. 定义状态：`dp[i]` 或 `dp[i][j]` 代表什么含义
2. 写出转移方程：如何从已知状态推出新状态
3. 确定初始值和遍历顺序

例题 11：最长递增子序列（LIS，线性 DP）

`dp[i]` = 以 `nums[i]` 结尾的最长递增子序列长度。

```
int lengthOfLIS(vector<int>& nums) {
    int n = nums.size(), ans = 1;
    vector<int> dp(n, 1);
    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++)
            if (nums[j] < nums[i]) dp[i] = max(dp[i], dp[j]+1);
        ans = max(ans, dp[i]);
    }
```

```
    return ans;
}
```

例题 12：0/1 背包

n 件物品，背包容量 W，每件只能选一次，求最大总价值。

```
int main() {
    int n, W; cin >> n >> W;
    vector<int> w(n), v(n);
    for (int i = 0; i < n; i++) cin >> w[i] >> v[i];

    vector<long long> dp(W+1, 0);
    for (int i = 0; i < n; i++)
        for (int j = W; j >= w[i]; j--)  // 逆序！保证每件只选一次
            dp[j] = max(dp[j], dp[j-w[i]] + v[i]);

    cout << dp[W] << endl;
}
```

例题 12.5：完全背包（每种物品可无限选）

将 0/1 背包的逆序改为正序即可：

```
for (int i = 0; i < n; i++)
    for (int j = w[i]; j <= W; j++)   // 正序！允许重复选同一件
        dp[j] = max(dp[j], dp[j-w[i]] + v[i]);
```

零钱兑换（完全背包的变形）：给定面额数组 coins，凑成 amount 的最少硬币数。

```
int coinChange(vector<int>& coins, int amount) {
    vector<int> dp(amount+1, INT_MAX);
    dp[0] = 0;
    for (int c : coins)
        for (int j = c; j <= amount; j++)
            if (dp[j-c] != INT_MAX)
                dp[j] = min(dp[j], dp[j-c] + 1);
    return dp[amount] == INT_MAX ? -1 : dp[amount];
}
```

例题 13：最长公共子序列（LCS，二维 DP）

```
int longestCommonSubsequence(string text1, string text2) {
    int m = text1.size(), n = text2.size();
    vector<vector<int>> dp(m+1, vector<int>(n+1, 0));
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++) {
            if (text1[i-1] == text2[j-1]) dp[i][j] = dp[i-1][j-1] + 1;
            else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        }
```

```
    return dp[m][n];
}
```

---

7. 图论（基础）

识别关键词

- 节点+边的关系网络
- 有向图执行顺序、判环（拓扑排序）
- 带权图最短路（Dijkstra）

子类型分析

子类型	使用场景	算法	**CSP 考频**
拓扑排序	有向无环图（DAG），求合法执行顺序，判断是否有环	Kahn（BFS 入度法）	⭐⭐ T2 级
Dijkstra	非负权有向图，单源最短路	最小堆 + 贪心松弛	⭐ T3 级

> 💡 **CSP 300** 分重点：拓扑排序掌握即可覆盖大多数图论 T2 题；Dijkstra 可选学。

例题 **14**：课程表（拓扑排序，**Kahn** □法）

判断有向图是否有环：若能拓扑排序完成，则无环。

```
bool canFinish(int n, vector<vector<int>>& pre) {
    vector<int> indegree(n, 0);
    vector<vector<int>> adj(n);
    for (auto& p : pre) {
        adj[p[1]].push_back(p[0]);
        indegree[p[0]]++;
    }
    queue<int> q;
    for (int i = 0; i < n; i++) if (indegree[i]==0) q.push(i);
    int cnt = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop(); cnt++;
        for (int v : adj[u]) if (--indegree[v]==0) q.push(v);
    }
    return cnt == n; // 处理了所有节点 = 无环
}
```

例题 **15**：**Dijkstra** 单源最短路

```
void dijkstra(int src, vector<vector<pair<int,long long>>>& adj,
              vector<long long>& dist) {
    int n = adj.size();
    dist.assign(n, LLONG_MAX);
    priority_queue<pair<long long,int>,
                   vector<pair<long long,int>>, greater<>> pq;
    dist[src] = 0; pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue; // 已过期的条目跳过
        for (auto [v, w] : adj[u])
            if (dist[u]+w < dist[v]) { dist[v]=dist[u]+w; pq.push({dist[v],v}); }
    }
}
```

> 📝 **CCF CSP** 真题：202203-2 训练计划（逆向拓扑排序求最晚开始时间）

---

8. 前缀和 / 差分

识别关键词

- 前缀和：多次查询区间和，数组静态不变
- 差分数组：多次对某区间批量加减，最后查询每个位置的值

子类型分析

子类型	使用场景	核心操作
一维前缀和	静态数组，频繁区间求和	`pre[i]=pre[i-1]+a[i]`；查询 `[l,r]` = `pre[r+1]-pre[l]`
差分数组	频繁区间整体加减，最后一次性还原	`d[l]+=v; d[r+1]-=v`，前缀和还原
二维前缀和	矩阵子区域求和	`pre[i][j]=a[i][j]+pre[i-1][j]+pre[i][j-1]-pre[i-1][j-1]`

例题 16：区间求和（一维前缀和）

```
// 预处理（O(n)）
vector<long long> prefix(n+1, 0);
for (int i = 0; i < n; i++) prefix[i+1] = prefix[i] + a[i];
```

```
// 查询 [l, r]（0-indexed）O(1)
long long query(int l, int r) { return prefix[r+1] - prefix[l]; }
```

例题 **17**：区间批量加（差分数组）

```
vector<long long> d(n+2, 0);
// 对 [l, r] 区间加 v（0-indexed）
void add(int l, int r, long long v) { d[l] += v; d[r+1] -= v; }

// 最后还原：对 d 求前缀和得到最终数组
long long cur = 0;
for (int i = 0; i < n; i++) { cur += d[i]; a[i] = cur; }
```

> 📝 **CCF CSP** 真题：202109-2 非零段划分（差分枚举最优阈值）· 202012-2 期末预测之最佳阈值（排序+前缀和枚举）

8.5 矩阵操作（重点技巧）

识别关键词

- 输入是二维数组，要求旋转、转置、翻转、边界遍历
- 题目强调“行列坐标变化”或“按层（圈）处理”
- 常见于 CSP T1/T2：代码不难，但容易写错下标

子类型分析

子类型	场景	易错点
矩阵转置	`a[i][j] -> b[j][i]`	行列边界写反
90°旋转	图像旋转、坐标变换	原地旋转时四元交换顺序
螺旋**/**边界遍历	按圈输出、模拟打印	四个边界更新时机
二维前缀和	多次子矩形求和	容斥公式符号
矩阵乘法（基础）	状态转移、线性变换	三重循环下标对应

例题 **17.5**：矩阵顺时针旋转 **90°**（**n×n**）

思路：先转置，再每一行反转。

```
void rotate90(vector<vector<int>>& a) {
    int n = a.size();
    // 1) 转置
```

```
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            swap(a[i][j], a[j][i]);
    // 2) 每行反转
    for (int i = 0; i < n; i++)
        reverse(a[i].begin(), a[i].end());
}
```

例题 17.6：二维前缀和（子矩形求和）

设 `pre[i][j]` 表示左上角 `(1,1)` 到 `(i,j)` 的元素和（1-indexed）：

```
// 构建 pre
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        pre[i][j] = a[i][j] + pre[i-1][j] + pre[i][j-1] - pre[i-1][j-1];

// 查询子矩形 (x1,y1) 到 (x2,y2)
long long query(int x1, int y1, int x2, int y2) {
    return pre[x2][y2] - pre[x1-1][y2] - pre[x2][y1-1] + pre[x1-1][y1-1];
}
```

9. 单调栈 / 单调队列

识别关键词

- 「每个位置左/右第一个比它更大/更小的元素」→ 单调栈
- 「大小为 k 的滑动窗口中的最大/最小值」→ 单调队列

子类型分析

子类型	使用场景	数据结构	单调方向
单调递减栈	每个元素右边第一个更大值	`stack<int>`（存下标）	栈底 → 栈顶值递减
单调递增栈	每个元素右边第一个更小值	`stack<int>`（存下标）	栈底 → 栈顶值递增
单调队列（deque）	大小为 k 的滑动窗口最大值	`deque<int>`（存下标）	队头最大，队尾最小

例题 18：每日温度（单调递减栈）

求每天需要等几天才会遇到更高温度。

```
vector<int> dailyTemperatures(vector<int>& T) {
    int n = T.size();
    vector<int> ans(n, 0);
    stack<int> stk; // 存下标，栈内温度单调递减
```

```
    for (int i = 0; i < n; i++) {
        while (!stk.empty() && T[i] > T[stk.top()]) {
            int j = stk.top(); stk.pop();
            ans[j] = i - j; // 找到了 j 右边第一个更大温度
        }
        stk.push(i);
    }
    return ans;
}
```

例题 19：滑动窗口最大值（单调队列）

```
vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    deque<int> dq; // 存下标，对应值单调递减
    vector<int> res;
    for (int i = 0; i < (int)nums.size(); i++) {
        while (!dq.empty() && dq.front() <= i-k) dq.pop_front(); // 过期出队
        while (!dq.empty() && nums[dq.back()] <= nums[i]) dq.pop_back();
        dq.push_back(i);
        if (i >= k-1) res.push_back(nums[dq.front()]); // 队头是窗口最大值
    }
    return res;
}
```

---

10. 并查集

识别关键词

- 「合并集合」「判断是否同属一组」
- 动态连通性（朋友的朋友是朋友）
- 联通分量数量变化

子类型分析

子类型	特征	适用场景
基础并查集	路径压缩 + 按秩合并，每次操作近似 O(1)	朋友圈、岛屿合并（替代 DFS）
带权并查集	额外维护节点间的关系权重	食物链、带方向的分组（CSP 300 较少考）

例题 20：朋友圈（并查集模板）

```
struct DSU {
    vector<int> parent, rank_;
    int components;
```

```
    DSU(int n) : parent(n), rank_(n, 0), components(n) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]); // 路径压缩
        return parent[x];
    }
    void unite(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return;
        if (rank_[px] < rank_[py]) swap(px, py); // 按秩合并
        parent[py] = px;
        if (rank_[px] == rank_[py]) rank_[px]++;
        components--;
    }
};
```

10.5 边界易错点 + 常用函数模板

> 这部分不是新算法，而是 最容易丢分的实现细节。建议考前反复看。

边界易错点清单（高频）

场景	容易写错	推荐写法
下标范围	`for (i=0; i<=n; i++)` 越界	`for (int i = 0; i < n; i++)`
二分边界	`while (l <= r)` 时更新不一致导致死循环	左闭右闭就配套 `mid=(l+r)/2` 与 `l=mid+1/r=mid-1`；左闭右开常用 `while (l < r)`
前缀和	把 `pre` 开成 `n` 导致 `pre[r+1]` 越界	`vector<long long> pre(n+1,0)`
差分	对 `d[r+1]` 写入越界	`d` 至少开 `n+2`
滑动窗口	忘记先弹出过期下标再取答案	固定顺序：过期出队 → 维护单调性 → 入队 → 取值
网格 DFS/BFS	`x/y` 越界判断不统一	封装 `inGrid(x,y,n,m)`，所有地方复用
整型溢出	`int` 乘法爆 32 位	涉及乘法/前缀和用 `long long`

常用函数模板（可直接套）

1）网格边界判断

```
inline bool inGrid(int x, int y, int n, int m) {
    return x >= 0 && x < n && y >= 0 && y < m;
}
```

2）左闭右开二分（找第一个 `>= target`）

```
int lowerBound(const vector<int>& a, int target) {
    int l = 0, r = (int)a.size(); // [l, r)
    while (l < r) {
        int mid = l + (r - l) / 2;
        if (a[mid] >= target) r = mid;
        else l = mid + 1;
    }
    return l; // 可能等于 a.size()
}
```

3）前缀和区间查询（0-indexed）

```
vector<long long> buildPrefix(const vector<int>& a) {
    int n = a.size();
    vector<long long> pre(n + 1, 0);
    for (int i = 0; i < n; i++) pre[i + 1] = pre[i] + a[i];
    return pre;
}

long long rangeSum(const vector<long long>& pre, int l, int r) {
    return pre[r + 1] - pre[l];
}
```

4）统一方向数组（网格四联通）

```
int dx[4] = {-1, 1, 0, 0};
int dy[4] = {0, 0, -1, 1};
```

5）复杂排序模板（二维数组/二元组）

> 需求：先按第一关键字升序；若第一关键字相同，再按第二关键字升序。

```
vector<vector<int>> a = {{2, 3}, {1, 5}, {2, 1}, {1, 2}};
sort(a.begin(), a.end(), [](const vector<int>& x, const vector<int>& y) {
    if (x[0] != y[0]) return x[0] < y[0]; // 第一关键字
```

```
    return x[1] < y[1];                   // 第二关键字
});
```

> 若是 `vector<pair<int,int>>`，可直接 `sort(v.begin(), v.end())`（默认按 first，再按 second 升序）。

---

11. STL 日常速查（非□法）

> 这章是“写题常用 API 备忘录”，偏工程/语法，不算算法□体。

1）`vector` / `string` 常用

- `vector` 适合动态数组；`resize` 会改变长度，`size()` 返回当前元素个数。
- `string` 支持和容器类似的迭代器操作，可直接配合 `reverse`、`sort`。

```
vector<int> a;
a.push_back(3);
a.pop_back();
a.resize(10, 0);
int n = a.size();

string s = "abc";
s += 'd';
reverse(s.begin(), s.end());
```

2）排序、去重、反转

- `unique` 只会把“相邻重复”挪到后面，所以通常要先 `sort` 再 `erase`。
- 自定义排序建议先明确“第一关键字/第二关键字”的优先级。

```
sort(a.begin(), a.end());
sort(a.begin(), a.end(), greater<int>()); // 降序

a.erase(unique(a.begin(), a.end()), a.end()); // 先排序再 unique
reverse(a.begin(), a.end());
```

3）二分与有序容器查询

- 这些 API 要求区间有序；返回的是迭代器，使用前注意是否等于 `end()`。

```
auto it1 = lower_bound(a.begin(), a.end(), x); // 第一个 >= x
auto it2 = upper_bound(a.begin(), a.end(), x); // 第一个 >  x
bool has = binary_search(a.begin(), a.end(), x);
```

4）`map / unordered_map` 统计频次

- `map` 有序（红黑树，`O(log n)`）；`unordered_map` 无序（哈希，均摊 `O(1)`）。

- 只判断是否存在时优先用 `count`，避免不必要的插入。

```
unordered_map<int, int> cnt;
for (int x : a) cnt[x]++;

if (cnt.count(5)) {
    // 键 5 存在
}
```

5）队列 / 栈 / 双端队列

- `queue` 先进先出，`stack` 后进先出；`deque` 支持两端插入删除。
- 访问前先判空：`if (!q.empty()) ...`，避免越界访问。

```
queue<int> q; q.push(1); q.front(); q.pop();
stack<int> st; st.push(1); st.top(); st.pop();
deque<int> dq; dq.push_back(1); dq.push_front(2);
```

6）优先队列（堆）

- 默认是大根堆；要小根堆就传 `greater<T>`。
- 适合“反复取当前最值并插入新值”的场景。

```
priority_queue<int> maxHeap; // 大根堆
priority_queue<int, vector<int>, greater<int>> minHeap; // 小根堆
```

7）常用数值工具

- `accumulate` 初始值写 `0LL` 可防止整型求和溢出。
- `max_element/min_element` 返回迭代器，记得解引用。

```
long long sum = accumulate(a.begin(), a.end(), 0LL);
int mx = *max_element(a.begin(), a.end());
int mn = *min_element(a.begin(), a.end());

iota(a.begin(), a.end(), 0); // 填充 0,1,2,...
```

8）字符操作 / 字符映射 / 字符转换 / ASCII

- 字符类别判断优先用 `isalpha/isdigit/islower/isupper`（记得 `#include <cctype>`）。
- `char` 与 ASCII 码可互转；字母大小写可用 `tolower/toupper`。
- 统计字符频次可用 `vector<int>(128)`（标准 ASCII）或 `unordered_map<char,int>`（更通用）。

```
string s = "aB9z";

for (char &c : s) {
    if (isalpha(c)) {
        c = tolower(c); // 统一转小写
    }
}
```

```
vector<int> cnt(128, 0);      // ASCII 频次
for (char c : s) cnt[(int)c]++;

int codeA = (int)'A';         // 65
char ch = (char)(codeA + 32); // 'a'

unordered_map<char, char> mp; // 字符映射
mp['('] = ')';
mp['['] = ']';
mp['{'] = '}';
```

各□法时间复杂度速查

算法	时间复杂度	适用数据规模
暴力模拟	$O(n^2)$	$n \le 3000$
二分查找	$O(\log n)$	$n \le 10^9$
二分答案	$O(n \log V)$	$n \le 10^6$
双指针	**$O(n)$**	**$n \le 10^6$**
贪心（排序后）	$O(n \log n)$	$n \le 10^6$
DFS / BFS	$O(V + E)$	$V,E \le 10^5$
线性 DP	$O(n) \sim O(n^2)$	$n \le 10^4$
背包 DP	$O(n \times W)$	$n \times W \le 10^7$
拓扑排序	$O(V + E)$	$V,E \le 10^5$
Dijkstra（堆）	$O((V+E) \log V)$	$V,E \le 10^5$
前缀和	$O(n)$ 预处理，$O(1)$ 查询	$n \le 10^7$
矩阵操作	$O(nm)$（转置/遍历），$O(n^3)$（乘法）	$n,m \le 500$（乘法更小）
单调栈/队列	$O(n)$	$n \le 10^6$
并查集	$O(\alpha(n)) \approx O(1)$	$n \le 10^6$

CSP 300 *300+*